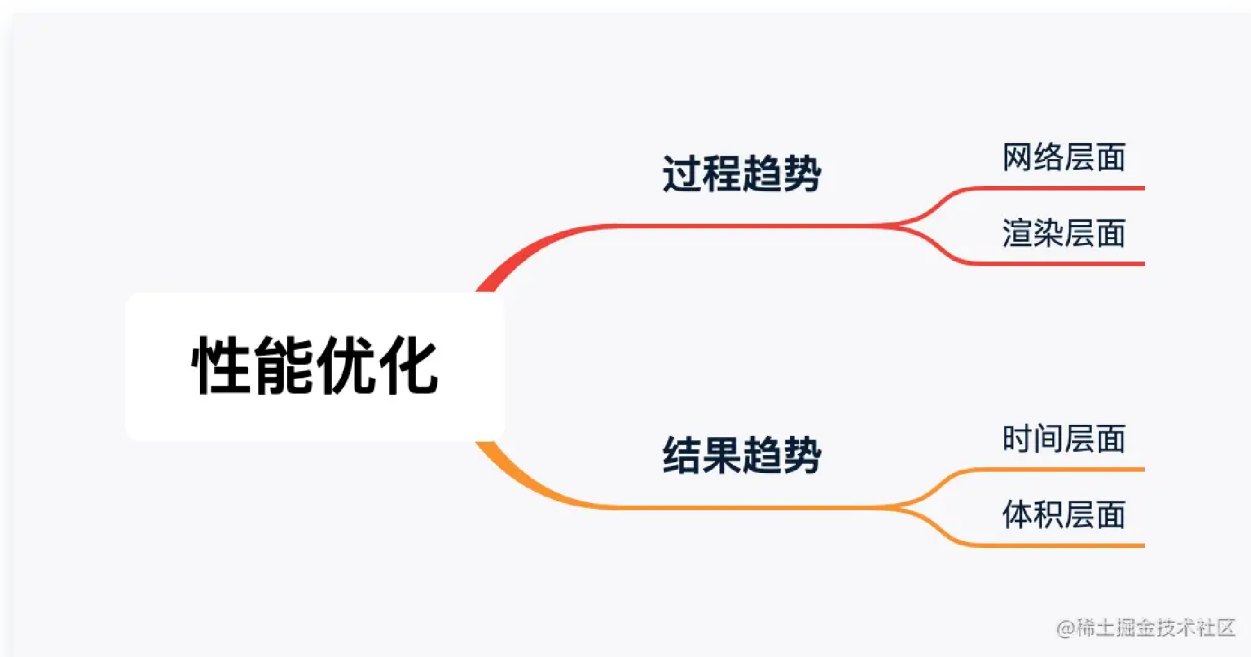


一.性能优化的指标



1.白屏时间

白屏也就是浏览器在渲染页面，如果白屏时间过长那么说明渲染的时间过长，一些常见的导致渲染时间过长的原因：

(1) HTML 代码中的 JavaScript 代码（简称 JS 代码）会阻断 DOM 树的构造，因为浏览器认为这段 JS 代码可能会修改 DOM 结构，所以必须等待 JS 代码执行完毕，再恢复 DOM 树的构造过程。这是由浏览器的安全解86157决定的，目前并没有指定某个 JS 代码不涉及 DOM 的属性。

(2) 浏览器必须等待样式表加载完成，才能开始构建 CSSOM 树。

(3) 还有一种特殊情况，浏览器在解析 HTML 时遇到 JS 代码，而此时 CSSOM 树还未构建完成，则浏览器会暂停脚本的执行（浏览器同时也会暂停继续向下解析 HTML 代码，从而导

(3) 还有一种特殊情况，浏览器在解析 HTML 时遇到 JS 代码，而此时 CSSOM 树还未构建完成，则浏览器会暂停脚本的执行（浏览器同时也会暂停继续向下解析 HTML 代码，从而导

致 DOM 树的构建过程被暂停阻塞)，直到 CSS 样式文件下载完成，并完成 CSSOM 树的构建，才会重新恢复原来的解析。这也是由浏览器的安全解析策略决定的。

第三种原因危害最大，理由：

通过对上面各因素的分析，我们可以发现，HTML 中的内联 JS 代码执行的危害之处，不在于它阻断了 DOM 树的构建过程，除非是一段特别恶劣的 JS 代码运行非常慢。通常内联的 JS 代码运行大概消耗几十毫秒，也就是暂停构建几毫秒到几十毫秒。它最大的危害在于上面第三种情况提到的，即 DOM 树构建被阻塞的时间不是只有 JS 代码运行的时间，而是会加上样式资源文件下载和 CSSOM 树的构建时间，这时浏览器所进行的串行解析过程，与我们在前面期望的 DOM 树和 CSSOM 树^①的并行解析过程相差甚远。我们用图 1-5 来表示这个过程。

2. 首屏时间

网页加载时用户首次看到的内容展示区域，也可以理解为用户在进入网页后所能立即看到的可视内容。

在 Chrome 浏览器中，你可以通过 Performance API 或者 Chrome 浏览器的开发者工具来查看这两个指标。

1. 使用 Performance API：

你可以通过编写 JavaScript 代码来使用 Performance API 来捕获这两个指标。以下是一个简单的示例代码：

```
performance.getEntriesByType('paint').forEach(entry =>
{
    console.log(entry.name + ' occurs at ' +
entry.startTime);
});
```

这段代码将打印出所有 paint 相关的指标，包括 First Paint 和 First Contentful Paint。

2. 使用 Chrome 开发者工具：

打开 Chrome 浏览器，按下 F12 键或右键点击页面并选择“检查”来打开开发者工具。然后选择“Performance”选项卡，在顶部的菜单中点击“Reload”按钮以重新加载页面并开始记录性能数据。在加载完成后，你可以在图表中找到 First Paint 和 First Contentful Paint 的时间点，也可以在 Summary 面板中找到这两个指标的数值。

这些方法可以帮助你 在 Chrome 浏览器中查看 First Paint 和 First Contentful Paint 这两个页面加载性能指标。

3. 页面整体加载完成时间

也叫做PageLoad,页面整体加载完成

window.onload 和 DOMContentLoaded

```
1 window.addEventListener('load', function () {
2     // 页面的全部资源加载完才会执行，包括图片、视频等
3 })
4 document.addEventListener('DOMContentLoaded', function () {
5     // DOM 渲染完即可执行，此时图片、视频还可能没有加载完
6 })
```

二.性能优化的实战

Souders 在《高性能网站建设指南》一书中总结了 12 条基本规则：

- 尽量减少 HTTP 请求。
- 使用 CDN。
- 静态资源使用 Cache。
- 启用 Gzip 压缩。
- JavaScript 脚本尽量放在页面底部。
- CSS 样式表放在顶部。
- 避免 CSS 表达式。
- 减少内联 JavaScript 和 CSS 的使用，尽可能使用外部的 JavaScript 和 CSS。
- 减少 DNS 查询。
- 精简 JavaScript。
- 避免重定向。
- 删除重复的脚本。

主要可以从两个层面去进行性能优化，分别是开发阶段和打包阶段。

开发阶段

1.SSR

SSR应用相比SPA应用更好的性能

2.图像

主要围绕 **图像类型** 做相关处理，同时也是接入成本较低的 **性能优化策略**。只需做到以下两点即可。

- **图像选型**：了解所有图像类型的特点及其何种应用场景最合适
- **图像压缩**：在部署到生产环境前使用工具或脚本对其压缩处理

图像选型 一定要知道每种图像类型的 **体积/质量/兼容/请求/压缩/透明/场景** 等参数相对值，这样才能迅速做出判断在何种场景使用何种类型的图像。

类型	体积	质量	兼容	请求	压缩	透明	场景
JPG	小	中	高	是	有损	不支持	背景图、轮播图、色彩丰富图
PNG	大	高	高	是	无损	支持	图标、透明图
SVG	小	高	高	是	无损	支持	图标、矢量图
WebP	小	中	低	是	兼备	支持	看兼容情况
Base64	看情况	中	高	否	无损	支持	图标

由于现在大部分 **webpack** 图像压缩工具不是安装失败就是各种环境问题，所以可以在发布项目到生产前使用图像压缩工具处理，这样运行稳定也不会增加打包时间。

好用的图像压缩工具对比：

工具	开源	收费	API	免费体验
QuickPicture	✗	✓	✗	可压缩类型较多，压缩质感较好，有体积限制，有数量限制
ShrinkMe	✗	✗	✗	可压缩类型较多，压缩质感一般，无数量限制，有体积限制
Squoosh	✓	✗	✓	可压缩类型较少，压缩质感一般，无数量限制，有体积限制
TinyJpg	✗	✓	✓	可压缩类型较少，压缩质感很好，有数量限制，有体积限制
TinyPng	✗	✓	✓	可压缩类型较少，压缩质感很好，有数量限制，有体积限制
Zhitu	✗	✗	✗	可压缩类型一般，压缩质感一般，有数量限制，有体积限制

图像策略 也许处理一张图像就能完爆所有 构建策略，因此是一种很廉价但极有效的 性能优化策略

3.CSS

- 避免出现超过三层的 嵌套规则
- 避免为 ID选择器 添加多余选择器
- 避免使用 标签选择器 代替 类选择器
- 避免使用 通配选择器，只对目标节点声明规则
- 避免重复匹配重复定义，关注 可继承属性
- 当然最重要的还是避免回流和重绘:

1>缓存 DOM计算属性

2>使用类合并样式，避免逐条改变样式

3>使用 display 控制 DOM显隐，将 DOM离线化

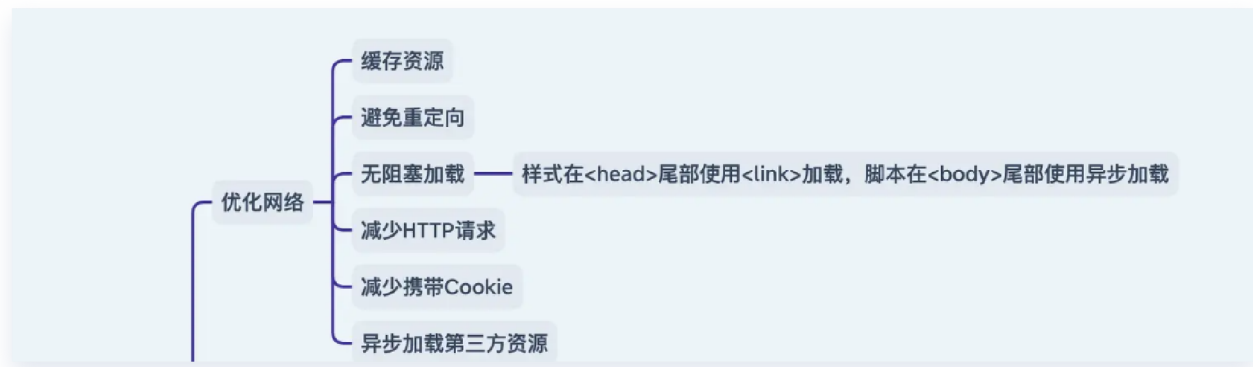
4.DOM

- 缓存 DOM计算属性
- 避免过多 DOM操作
- 使用 DOMFragment 缓存批量化 DOM操作

5.阻塞

- 脚本与 DOM/其它脚本 的依赖关系很强：对 `<script>` 设置 `defer`
- 脚本与 DOM/其它脚本 的依赖关系不强：对 `<script>` 设置 `async`

6.网络



除此之外，还有CDN的使用以及DNS预解析：

内容分发网络简称**CDN**，指一组分布在各地存储数据副本并可根据就近原则满足数据请求的服务器。其核心特征是 **缓存** 和 **回源**，缓存是把资源复制到 **CDN服务器** 里，回源是 **资源过期/不存在** 就向上层服务器请求并复制到 **CDN服务器** 里。

使用 **CDN** 可降低网络拥塞，提高用户访问响应速度和命中率。构建在现有网络基础上的智能虚拟网络，依靠部署在各地服务器，通过中心平台的调度、负载均衡、内容分发等功能模块，使用户就近获取所需资源，这就是 **CDN** 的终极使命。

基于 **CDN** 的**就近原则**所带来的优点，可将网站所有静态资源全部部署到 **CDN服务器** 里。那静态资源包括哪些文件？通常来说就是无需服务器产生计算就能得到的资源，例如不常变化的 **样式文件**、**脚本文件** 和 **多媒体文件**(**字体/图像/音频/视频**) 等。

DNS预解析是一种优化技术，通过提前获取域名对应的IP地址，可以加速页面加载速度。当我们在网页中使用 **<link>** 标签的 **rel** 属性设置为 **dns-prefetch** 时，浏览器会提前进行 DNS 解析，以便在需要时能够更快地建立连接。

7.渲染层面



8. 框架层面

以react为例，可以进行的性能优化可以从以下几个方面入手：(具体细节和原理已经在react八股中详细说明，此处只进行总结)

(1) 路由懒加载

需要结合Suspense组件，react.lazy()函数来使用

(2) 函数组件用React.memo()包裹，类组件继承pureComponent

(3) 用useMemo缓存计算结果

(4) 用useCallback缓存函数

9. 长列表优化

(1) 时间分片

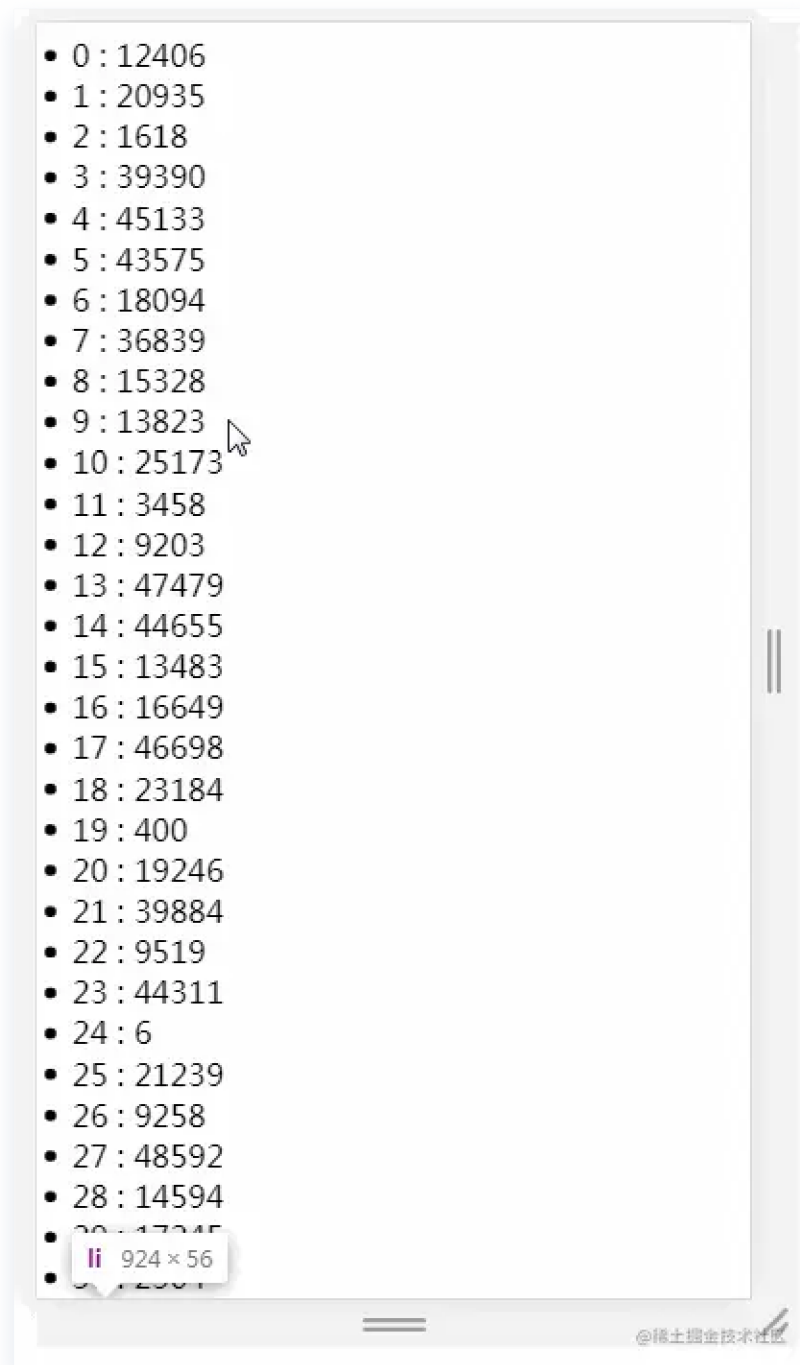
A.使用定时器

长列表渲染时页面的卡顿是由于同时渲染大量DOM所引起的，所以我们考虑将渲染过程分批进行

在这里，我们使用 `setTimeout` 来实现分批渲染

```
//需要插入的容器
let ul = document.getElementById('container');
// 插入十万条数据
let total = 100000;
// 一次插入 20 条
let once = 20;
//总页数
let page = total/once
//每条记录的索引
let index = 0;
//循环加载数据
function loop(curTotal,curIndex){
    if(curTotal ≤ 0){
        return false;
    }
    //每页多少条
    let pageCount = Math.min(curTotal , once);
    setTimeout(()⇒{
        for(let i = 0; i < pageCount; i++){
            let li = document.createElement('li');
            li.innerText = curIndex + i + ' : ' + ~
(Math.random() * total)
            ul.appendChild(li)
        }
        loop(curTotal - pageCount,curIndex + pageCount)
    },0)
}
loop(total,index);
```

用一个gif图来看一下效果



我们可以看到，页面加载的时间已经非常快了，每次刷新时可以很快的看到第一屏的所有数据，但是当我们快速滚动页面的时候，会发现页面出现闪屏或白屏的现象

为什么会出现闪屏现象呢

首先，理清一些概念。FPS 表示的是每秒钟画面更新次数。我们平时所看到的连续画面都是由一幅幅静止画面组成的，每幅画面称为一帧，FPS 是描述帧变化速度的物理量。

大多数电脑显示器的刷新频率是60Hz，大概相当于每秒钟重绘60次，FPS 为60frame/s，为这个值的设定受屏幕分辨率、屏幕尺寸和显卡的影响。

因此，当你对着电脑屏幕什么也不做的情况下，大多显示器也会以每秒60次的频率正在不断的更新屏幕上的图像。

为什么你感觉不到这个变化？

那是因为人的眼睛有视觉停留效应，即前一副画面留在大脑的印象还没消失，紧接着后一副画面就跟上来了，这中间只间隔了16.7ms($1000/60 \approx 16.7$)，所以会让你误以为屏幕上的图像是静止不动的。

而屏幕给你的这种感觉是对的，试想一下，如果刷新频率变成1次/秒，屏幕上的图像就会出现严重的闪烁，这样就很容易引起眼睛疲劳、酸痛和头晕目眩等症状。

大多数浏览器都会对重绘操作加以限制，不超过显示器的重绘频率，因为即使超过那个频率用户体验也不会有提升。因此，最平滑动画的最佳间隔是 $1000\text{ms}/60$ ，约等于16.6ms。

- `setTimeout` 的执行时间并不是确定的。在JS中，`setTimeout` 任务被放进事件队列中，只有主线程执行完才会去检查事件队列中的任务是否需要执行，因此 `setTimeout` 的实际执行时间可能会比其设定的时间晚一些。
- 刷新频率受屏幕分辨率和屏幕尺寸的影响，因此不同设备的刷新频率可能会不同，而 `setTimeout` 只能设置一个固定时间间隔，这个时间不一定和屏幕的刷新时间相同。

以上两种情况都会导致`setTimeout`的执行步调和屏幕的刷新步调不一致。

在 `setTimeout` 中对dom进行操作，必须要等到屏幕下次绘制时才能更新到屏幕上，如果两者步调不一致，就可能导致中间某一帧的操作被跨越过去，而直接更新下一帧的元素，从而导致丢帧现象。

B.使用 `requestAnimationFrame`

与 `setTimeout` 相比，`requestAnimationFrame` 最大的优势是由系统来决定回调函数的执行时机。

如果屏幕刷新率是60Hz,那么回调函数就每16.7ms被执行一次，如果刷新率是75Hz，那么这个时间间隔就变成了 $1000/75=13.3\text{ms}$ ，换句话说就是，`requestAnimationFrame` 的步伐跟着系统的刷新步伐走。它能保证回调函数在屏幕每一次的刷新闻隔中只被执行一次，这样就不会引起丢帧现象。

我们使用 `requestAnimationFrame` 来进行分批渲染：

```
// 需要插入的容器
let ul = document.getElementById('container');
// 插入十万条数据
let total = 100000;
// 一次插入 20 条
let once = 20;
// 总页数
let page = total/once
// 每条记录的索引
let index = 0;
// 循环加载数据
function loop(curTotal,curIndex){
    if(curTotal ≤ 0){
        return false;
    }
    // 每页多少条
    let pageCount = Math.min(curTotal , once);
    window.requestAnimationFrame(function(){
        for(let i = 0; i < pageCount; i++){
            let li = document.createElement('li');
```

```
        li.innerText = curIndex + i + ' : ' + ~
(Math.random() * total)
        ul.appendChild(li)
    }
    loop(curTotal - pageCount, curIndex + pageCount)
  })
}
loop(total, index);
```

看下效果

- 0: 88445
- 1: 43587
- 2: 35174
- 3: 19921
- 4: 63587
- 5: 22853
- 6: 3304
- 7: 73435
- 8: 1299
- 9: 97840
- 10: 73975
- 11: 76882
- 12: 59557
- 13: 22175
- 14: 70446
- 15: 50901
- 16: 34227
- 17: 82423
- 18: 92747
- 19: 24632
- 20: 75986
- 21: 31343
- 22: 17229
- 23: 66900
- 24: 777
- 25: 13735
- 26: 27701
- 27: 21235
- 28: 67435
- 29: 63553
- 30: 15681

我们可以看到，页面加载的速度很快，并且滚动的时候，也很流畅没有出现闪烁丢帧的现象。

C.使用 DocumentFragment

先解释一下什么是 DocumentFragment，文献引用自[MDN](#)

DocumentFragment，文档片段接口，表示一个没有父级文件的最小文档对象。它被作为一个轻量版的 Document 使用，用于存储已排好版的或尚未打好格式的XML片段。最大的区别是因为 DocumentFragment 不是真实 DOM 树的一部分，它的变化不会触发 DOM 树的（重新渲染），且不会导致性能等问题。

可以使用 document.createDocumentFragment 方法或者构造函数来创建一个空的 DocumentFragment

从MDN的说明中，我们得知 DocumentFragments 是DOM节点，但并不是DOM树的一部分，可以认为是存在内存中的，所以将子元素插入到文档片段时不会引起页面回流。

当 append 元素到 document 中时，被 append 进去的元素的样式表的计算是同步发生的，此时调用 getComputedStyle 可以得到样式的计算值。而 append 元素到 documentFragment 中时，是不会计算元素的样式表，所以 documentFragment 性能更优。当然现在浏览器的优化已经做的很好了，当 append 元素到 document 中后，没有访问 getComputedStyle 之类的方法时，现代浏览器也可以把样式表的计算推迟到脚本执行之后。

最后修改代码如下：

```
// 需要插入的容器
let ul = document.getElementById('container');
// 插入十万条数据
let total = 100000;
// 一次插入 20 条
```

```

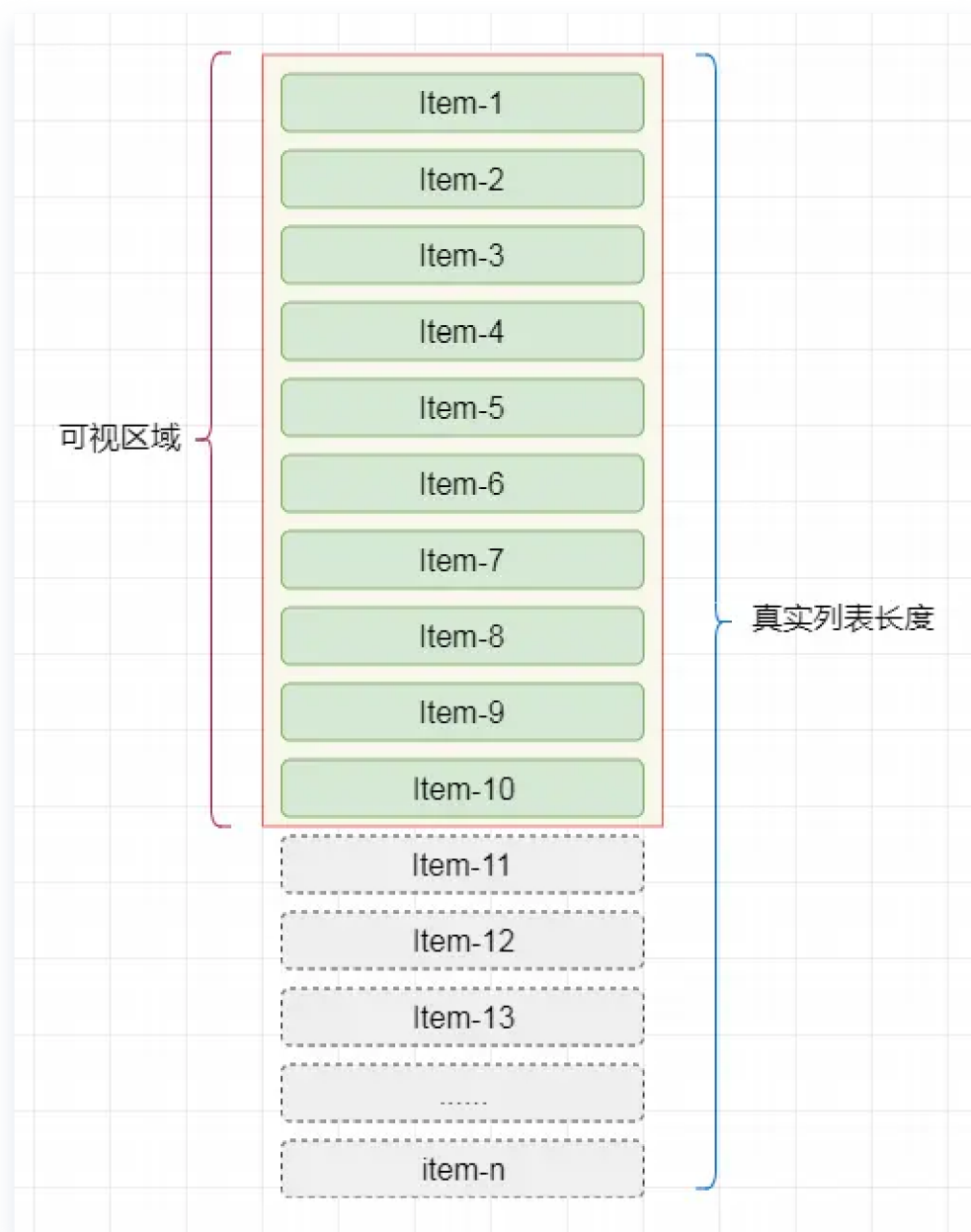
let once = 20;
//总页数
let page = total/once
//每条记录的索引
let index = 0;
//循环加载数据
function loop(curTotal,curIndex){
    if(curTotal ≤ 0){
        return false;
    }
    //每页多少条
    let pageCount = Math.min(curTotal , once);
    window.requestAnimationFrame(function(){
        let fragment = document.createDocumentFragment();
        for(let i = 0; i < pageCount; i++){
            let li = document.createElement('li');
            li.innerText = curIndex + i + ' : ' + ~
(Math.random() * total)
            fragment.appendChild(li)
        }
        ul.appendChild(fragment)
        loop(curTotal - pageCount,curIndex + pageCount)
    })
}
loop(total,index);

```

(2)虚拟列表

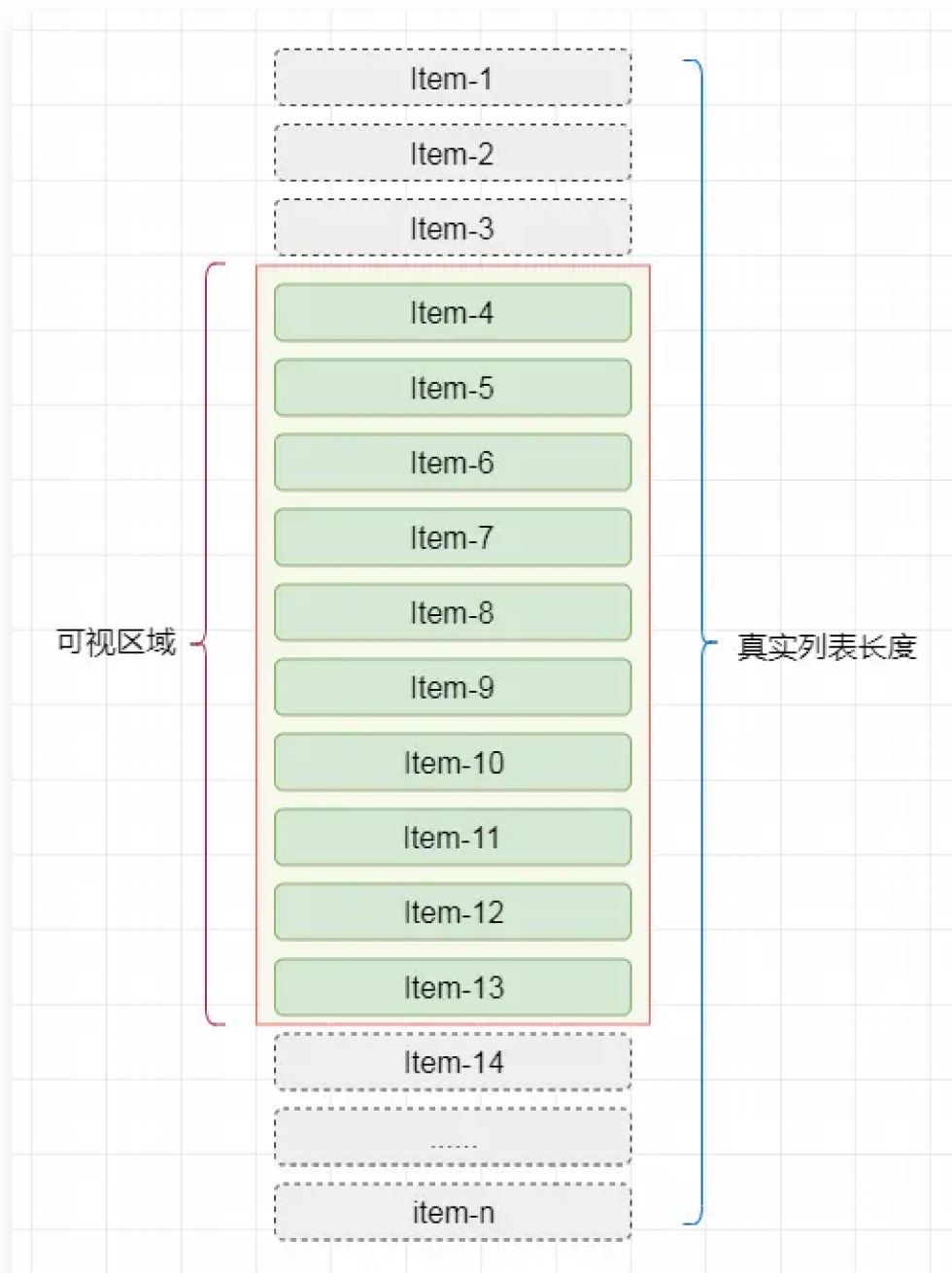
虚拟列表 其实是按需显示的一种实现，即只对 **可见区域** 进行渲染，对 **非可见区域** 中的数据不渲染或部分渲染的技术，从而达到极高的渲染性能。

假设有1万条记录需要同时渲染，我们屏幕的 **可见区域** 的高度为 **500px** ,而列表项的高度为 **50px** ，则此时我们在屏幕中最多只能看到10个列表项，那么在首次渲染的时候，我们只需加载10条即可。



说完首次加载，再分析一下当滚动发生时，我们可以通过计算当前滚动值得知此时在屏幕 **可见区域** 应该显示的列表项。

假设滚动发生，滚动条距顶部的位置为 **150px** ,则我们可得知在 **可见区域** 内的列表项为 **第4项** 至`第13项。

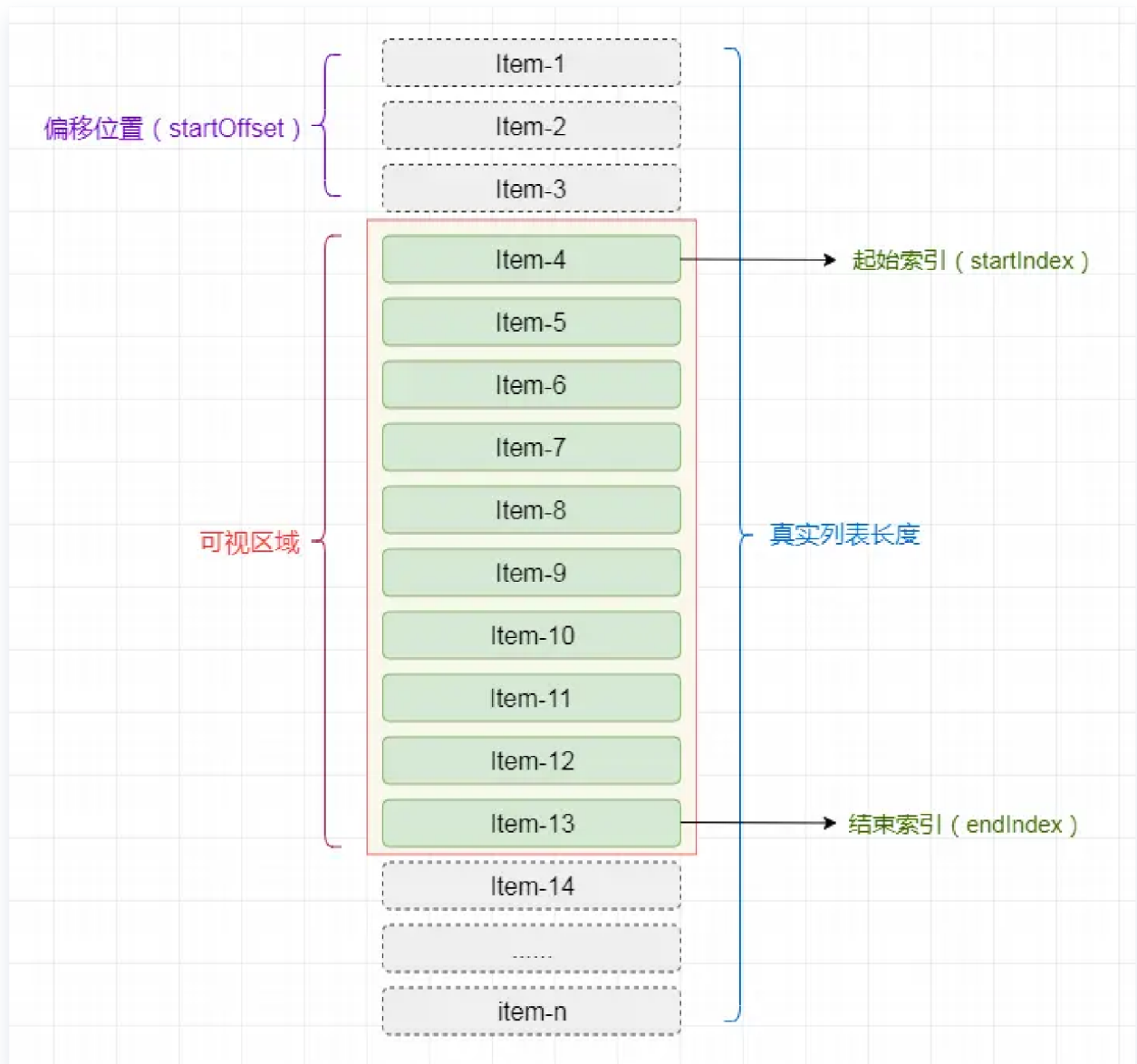


实现：

虚拟列表的实现，实际上就是在首屏加载的时候，只加载 **可视区域** 内需要的列表项，当滚动发生时，动态通过计算获得 **可视区域** 内的列表项，并将 **非可视区域** 内存在的列表项删除。

- 计算当前 **可视区域** 起始数据索引(`startIndex`)
- 计算当前 **可视区域** 结束数据索引(`endIndex`)
- 计算当前 **可视区域的** 数据，并渲染到页面中

- 计算 `startIndex` 对应的数据在整个列表中的偏移位置 `startOffset` 并设置到列表上



由于只是对 **可视区域** 内的列表项进行渲染，所以为了保持列表容器的高度并可正常的触发滚动，将Html结构设计成如下结构：

```

<div class="infinite-list-container">
  <div class="infinite-list-phantom"></div>
  <div class="infinite-list">
    <!-- item-1 -->
    <!-- item-2 -->
    <!-- ..... -->
    <!-- item-n -->
  </div>
</div>

```

- `infinite-list-container` 为 **可视区域** 的容器
- `infinite-list-phantom` 为容器内的占位，高度为总列表高度，用于形成滚动条
- `infinite-list` 为列表项的 **渲染区域**

接着，监听 `infinite-list-container` 的 `scroll` 事件，获取滚动位置 `scrollTop`

- 假定 **可视区域** 高度固定，称之为 `screenHeight`
- 假定 **列表每项** 高度固定，称之为 `itemSize`
- 假定 **列表数据** 称之为 `listData`
- 假定 **当前滚动位置** 称之为 `scrollTop`

则可推算出：

- 列表总高度 `listHeight` = `listData.length * itemSize`
- 可显示的列表项数 `visibleCount` = `Math.ceil(screenHeight / itemSize)`
- 数据的起始索引 `startIndex` = `Math.floor(scrollTop / itemSize)`
- 数据的结束索引 `endIndex` = `startIndex + visibleCount`
- 列表显示数据为 `visibleData` = `listData.slice(startIndex, endIndex)`

当滚动后，由于 **渲染区域** 相对于 **可视区域** 已经发生了偏移，此时我需要获取一个偏移量 `startOffset`，通过样式控制将 **渲染区域** 偏移至 **可视区域** 中。

- 偏移量 `startOffset` = `scrollTop - (scrollTop % itemSize);`

9.骨架屏对白屏的优化

平常在我们点进一个网站的时候，或许会看见一些动态的灰色条状物，他们代替着本应该出现在这里的内容，作为一种**加载态**的展现方式而呈现。而这里的灰色块，便是**骨架屏**，现在绝大多数的大厂的网站都会用骨架屏的方案来代替原先的转圈loading了，虽然都是加载态的一种方式，但是相较于干巴巴的转圈，这种在页面数据尚未加载前先给用户展示出页面的大致结构，直到请求数据返回后再渲染页面，填充好本应该展现的内容，给人一种**创造即时转换的错觉**

骨架屏的原理

在进行实际项目开发的时候，一般会让你实现**空态**和**加载态**，后端会给你一个请求接口返回的参数，比如这里我们将 `sending` 作为请求完接口返回的参数，当 `sending=0` 的时候，我们这时候就可以返回**空态**，也就是我们常见的报错页面，这些一般需要我们单独去写一份的；当 `sending=1` 的时候，我们则进入**加载态**，也就是我们常见的加载页面，当资源请求完成后再进行页面的替换即可，这也就是骨架屏的基本实现原理

目前市面上的骨架屏方案大致有三种：

- 手动维护骨架屏的代码
- 使用图片作为骨架屏
- 自动生成骨架屏

手动维护骨架屏的代码的优势是**可定制化高**，但相对的缺点就是这无疑会增加开发者的心智负担；而使用图片作为骨架屏，虽然也比较的耗费人力，但和开发的关系不大了

打包阶段

主要围绕 `webpack` 做相关处理，同时也是接入最普遍的**性能优化策略**。其他构建工具的处理也是大同小异，可能只是配置上不一致。说到 `webpack` 的**性能优化**，无疑是从**时间层面**和**体积层面**入手。

 表示 **减少打包时间**， 表示 **减少打包体积**。

- 减少打包时间： 缩减范围、 缓存副本、 定向搜索、 并行构建、 可视结构
- 减少打包体积： 分割代码、 摇树优化、 动态垫片、 按需加载、 作用提升、 压缩资源

缩减范围

配置`include/exclude`缩小Loader对文件的搜索范围，好处是 避免不必要的转译。`node_modules`目录的体积这么大，那得增加多少时间成本去检索所有文件啊？

`include/exclude` 通常在各大 Loader 里配置，`src`目录 通常作为源码目录，可做如下处理。当然 `include/exclude` 可根据实际情况修改。

```
export default {
  // ...
  module: {
    rules: [{
      exclude: /node_modules/,
      include: /src/,
      test: /\.js$/,
      use: "babel-loader"
    }]
  }
};
```

缓存副本

配置`cache`缓存Loader对文件的编译副本，好处是 再次编译时只编译修改过的文件。未修改过的文件干嘛要随着修改过的文件重新编译呢？

大部分 Loader/Plugin 都会提供一个可使用编译缓存的选项，通常包含 `cache` 字眼。以 `babel-loader` 和 `eslint-webpack-plugin` 为例。

```
import EslintPlugin from "eslint-webpack-plugin";

export default {
  // ...
  module: {
    rules: [{
      // ...
      test: /\.js$/,
      use: [{
        loader: "babel-loader",
        options: { cacheDirectory: true }
      }]
    }]
  },
  plugins: [
    new EslintPlugin({ cache: true })
  ]
};
```

定向搜索

配置`resolve`提高文件的搜索速度，好处是 `定向指定必须文件路径`。若某些第三方库以常规形式引入可能报错或希望程序自动索引特定类型文件都可通过该方式解决。

`alias` 映射模块路径，`extensions` 表明文件后缀，`noParse` 过滤无依赖文件。通常配置 `alias` 和 `extensions` 就足够。

```

export default {
  // ...
  resolve: {
    alias: {
      "#": AbsPath(""), // 根目录快捷方式
      "@": AbsPath("src"), // src目录快捷方式
      swiper: "swiper/js/swiper.min.js"
    }, // 模块导入快捷方式
    extensions: [".js", ".ts", ".jsx", ".tsx", ".json",
      ".vue"] // import路径时文件可省略后缀名
  }
};

```

并行构建

配置Thread将Loader单进程转换为多进程，好处是释放CPU多核并发的优势。在使用 webpack 构建项目时会有大量文件需解析和处理，构建过程是计算密集型的操作，随着文件增多会使构建过程变得越慢。

运行在 Node 里的 webpack 是单线程模型，简单来说就是 webpack 待处理的任務需一件件处理，不能同一时刻处理多件任务。

文件读写 与 计算操作 无法避免，能不能让 webpack 同一时刻处理多个任务，发挥多核 CPU 电脑的威力以提升构建速度呢？[thread-loader](#)来帮你，根据 CPU 个数开启线程。

在此需注意一个问题，若项目文件不算多就不要使用该 [性能优化建议](#)，毕竟开启多个线程也会存在性能开销。

```

import Os from "os";

export default {
  // ...
  module: {

```

```

    rules: [{
      // ...
      test: /\.js$/,
      use: [{
        loader: "thread-loader",
        options: { workers: Os.cpus().length }
      }, {
        loader: "babel-loader",
        options: { cacheDirectory: true }
      }]
    }]
  }
};

```

可视结构

配置 **BundleAnalyzer** 分析打包文件结构，好处是 **找出导致体积过大的原因**。从而通过分析原因得出优化方案减少构建时间。 **BundleAnalyzer** 是 **webpack** 官方插件，可直观分析 **打包文件** 的模块组成部分、模块体积占比、模块包含关系、模块依赖关系、文件是否重复、压缩体积对比等可视化数据。

可使用 [webpack-bundle-analyzer](#) 配置，有了它，我们就能快速找到相关问题。

```

import { BundleAnalyzerPlugin } from "webpack-bundle-analyzer";

export default {
  // ...
  plugins: [
    // ...
    BundleAnalyzerPlugin()
  ]
};

```


分割代码

分割各个模块代码，提取相同部分代码，好处是减少重复代码的出现频率。webpack v4 使用 `splitChunks` 替代 `CommonsChunksPlugin` 实现代码分割。

`splitChunks` 配置较多，详情可参考[官网](#)，在此笔者贴上常用配置。

```
export default {
  // ...
  optimization: {
    runtimeChunk: { name: "manifest" }, // 抽离
    // WebpackRuntime函数
    splitChunks: {
      cacheGroups: {
        common: {
          minChunks: 2,
          name: "common",
          priority: 5,
          reuseExistingChunk: true, // 重用已存在代码块
          test: AbsPath("src")
        },
        vendor: {
          chunks: "initial", // 代码分割类型
          name: "vendor", // 代码块名称
          priority: 10, // 优先级
          test: /node_modules/ // 校验文件正则表达式
        }
      }, // 缓存组
      chunks: "all" // 代码分割类型: all全部模块, async异步模
    }, // 代码块分割
  },
};
```

摇树优化

删除项目中未被引用代码，好处是 移除重复代码和未使用代码。摇树优化 首次出现于 `rollup`，是 `rollup` 的核心概念，后来在 `webpack v2` 里借鉴过来使用。

摇树优化 只对 `ESM规范` 生效，对其他模块规范失效。摇树优化 针对静态结构分析，只有 `import/export` 才能提供静态的 导入/导出 功能。因此在编写业务代码时必须使用 `ESM规范` 才能让 摇树优化 移除重复代码和未使用代码。

在 `webpack` 里只需将打包环境设置成 生产环境 就能让 摇树优化 生效，同时业务代码使用 `ESM规范` 编写，使用 `import` 导入模块，使用 `export` 导出模块。

```
export default {  
  // ...  
  mode: "production"  
};
```

动态垫片

通过垫片服务根据UA返回当前浏览器代码垫片，好处是 无需将繁重的代码垫片打包进去。每次构建都配置 `@babel/preset-env` 和 `core-js` 根据某些需求将 `Polyfill` 打包进来，这无疑又为代码体积增加了贡献。

`@babel/preset-env` 提供的 `useBuiltIns` 可按需导入 `Polyfill`。

- `false`: 无视 `target.browsers` 将所有 `Polyfill` 加载进来
- `entry`: 根据 `target.browsers` 将部分 `Polyfill` 加载进来(仅引入有浏览器不支持的 `Polyfill`，需在入口文件 `import "core-js/stable"`)
- `usage`: 根据 `target.browsers` 和检测代码里ES6的使用情况将部分 `Polyfill` 加载进来(无需在入口文件 `import "core-js/stable"`)

在此推荐大家使用 **动态垫片**。**动态垫片** 可根据浏览器 **UserAgent** 返回当前浏览器 **Polyfill**，其思路是根据浏览器的 **UserAgent** 从 **browserlist** 找出当前浏览器哪些特性缺乏支持从而返回这些特性的 **Polyfill**。对这方面感兴趣的同学可参考[polyfill-library](https://github.com/polyfill-library)和[polyfill-service](https://github.com/polyfill-service)的源码。

在此提供两个 **动态垫片** 服务，可在不同浏览器里点击以下链接看看输出不同的 **Polyfill**。

- **官方CDN服务**: polyfill.io/v3/polyfill...
- **阿里CDN服务**: polyfill.alicdn.com/polyfill.mi...

使用[html-webpack-tags-plugin](https://github.com/html-webpack/html-webpack-tags-plugin)在打包时自动插入 **动态垫片**。

```
import HtmlTagsPlugin from "html-webpack-tags-plugin";

export default {
  plugins: [
    new HtmlTagsPlugin({
      append: false, // 在生成资源后插入
      publicPath: false, // 使用公共路径
      tags:
        ["https://polyfill.alicdn.com/polyfill.min.js"] // 资源路径
    })
  ]
};
```

按需加载

将路由页面/触发性功能单独打包为一个文件，使用时才加载，好处是 **减轻首屏渲染的负担**。因为项目功能越多其打包体积越大，导致首屏渲染速度越慢。

首屏渲染时只需对应 **JS代码** 而无需其他 **JS代码**，所以可使用 **按需加载**。**webpack v4** 提供模块按需切割加载功能，配合 **import()** 可做到首屏渲染减包的效果，从而加快首屏渲染速度。只有当触发某些功能时才会加载当前功能的 **JS代码**。

`webpack v4` 提供魔术注解命名 `切割模块`，若无注解则切割出来的模块无法分辨出属于哪个业务模块，所以一般都是一个业务模块共用一个 `切割模块` 的注解名称。

```
const Login = () => import( /* webpackChunkName: "login" */
  "../..views/login");
const Logon = () => import( /* webpackChunkName: "logon" */
  "../..views/logon");
```

运行起来控制台可能会报错，在 `package.json` 的 `babel` 相关配置里接入 [@babel/plugin-syntax-dynamic-import](#) 即可。

```
{
  // ...
  "babel": {
    // ...
    "plugins": [
      // ...
      "@babel/plugin-syntax-dynamic-import"
    ]
  }
}
```

作用提升

分析模块间依赖关系，把打包好的模块合并到一个函数中，好处是 `减少函数声明和内存花销`。`作用提升` 首次出现于 `rollup`，是 `rollup` 的核心概念，后来在 `webpack v3` 里借鉴过来使用。

在未开启 `作用提升` 前，构建后的代码会存在大量函数闭包。由于模块依赖，通过 `webpack` 打包后会转换成 `IIFE`，大量函数闭包包裹代码会导致打包体积增大(`模块越多越明显`)。在运行代码时创建的函数作用域变多，从而导致更大的内存开销。

在开启 **作用提升** 后，构建后的代码会按照引入顺序放到一个函数作用域里，通过适当重命名某些变量以防止变量名冲突，从而减少函数声明和内存花销。

在 **webpack** 里只需将打包环境设置成 **生产环境** 就能让 **作用提升** 生效，或显式设置 **concatenateModules**。

```
export default {
  // ...
  mode: "production"
};
// 显式设置
export default {
  // ...
  optimization: {
    // ...
    concatenateModules: true
  }
};
```

压缩资源

压缩HTML/CSS/JS代码，压缩字体/图像/音频/视频，好处是 **更有效减少打包体积**。极致地优化代码都有可能不及优化一个资源文件的体积更有效。

针对 **HTML** 代码，使用[html-webpack-plugin](#)开启压缩功能。

```
import HtmlWebpackPlugin from "html-webpack-plugin";

export default {
  // ...
  plugins: [
    // ...
    HtmlWebpackPlugin({
      // ...
      minify: {
```

```

        collapseWhitespace: true,
        removeComments: true
    } // 压缩HTML
})
]
};

```

针对 CSS/JS 代码，分别使用以下插件开启压缩功能。其中 `OptimizeCss` 基于 `cssnano` 封装，`Uglifyjs` 和 `Terser` 都是 `webpack` 官方插件，同时需注意压缩 JS代码 需区分 ES5 和 ES6。

- [optimize-css-assets-webpack-plugin](#)：压缩 CSS代码
- [uglifyjs-webpack-plugin](#)：压缩 ES5 版本的 JS代码
- [terser-webpack-plugin](#)：压缩 ES6 版本的 JS代码

```

import OptimizeCssAssetsPlugin from "optimize-css-assets-
webpack-plugin";
import TerserPlugin from "terser-webpack-plugin";
import UglifyJsPlugin from "uglifyjs-webpack-plugin";

const compressOpts = type => ({
    cache: true, // 缓存文件
    parallel: true, // 并行处理
    [`_${type}Options`]: {
        beautify: false,
        compress: { drop_console: true }
    } // 压缩配置
});

const compressCss = new OptimizeCssAssetsPlugin({
    cssProcessorOptions: {
        autoprefixer: { remove: false }, // 设置autoprefixer保
        留过时样式
        safe: true // 避免cssnano重新计算z-index
    }
});

const compressJs = USE_ES6

```

```
? new TerserPlugin(compressOpts("terser"))
: new UglifyjsPlugin(compressOpts("uglify"));

export default {
  // ...
  optimization: {
    // ...
    minimizer: [compressCss, compressJs] // 代码压缩
  }
};
```

三.网站性能测试工具

chrome开发者工具中的performance各个指标:

1. **Parse HTML（解析HTML）**: 这个阶段是浏览器解析HTML代码并构建DOM树的过程。浏览器会逐行解析HTML代码，识别标签、属性等内容，并构建出DOM树结构。
2. **Scripting（脚本执行）**: 在这个阶段，浏览器会执行页面中的JavaScript代码，这些代码可能会修改DOM结构、样式等内容，从而影响页面的最终呈现。
3. **Recalculate Style and Layout（重新计算样式和布局）**: 当页面的DOM结构发生变化或者样式发生改变时，浏览器会重新计算元素的样式和布局，确保页面元素按照最新的规则进行排列和显示。
4. **Paint（绘制）**: 最后一个阶段是将页面元素绘制到屏幕上，包括文字、图像等内容的绘制过程。浏览器会根据之前计算好的样式和布局信息，将页面元素以正确的顺序绘制到屏幕上，呈现给用户。

其它的工具:

LightHouse 是一种开源的自动化工具，由 Google 创建和维护，用于改进网页质量。它通过运行一系列针对网页性能、可访问性、最佳实践和搜索引擎优化的测试，来评估网页的性能和用户体验。LightHouse 提供了网页性能的综合报告，并提供改进建议，以帮助开发人员优化其网站，提升用户体验。

LightHouse 的主要功能包括：

1. **性能评估**：评估网页的加载性能，包括首次内容渲染时间、总体加载时间、各种资源的加载时间等。
2. **可访问性分析**：检查网页的可访问性，包括对盲人、色盲和其他残障用户的友好程度，以及使用正确的 HTML 标记、语义化的内容等。
3. **最佳实践审查**：检查网页是否符合最佳实践，如使用安全的 HTTPS 连接、避免重定向、使用适当的缓存策略等。
4. **SEO 分析**：评估网页的搜索引擎优化情况，包括页面是否具有描述性标题、META 描述、良好的网页结构等。

你可以通过多种方式使用 LightHouse 工具：

- **Chrome DevTools**：LightHouse 作为 Chrome DevTools 的一部分提供。你可以在 Chrome 浏览器中打开 DevTools，选择“Audits”选项卡，然后运行 LightHouse 分析。
- **命令行**：你还可以使用 Node.js 命令行工具来运行 LightHouse。通过安装 LightHouse npm 包，你可以在终端中运行命令来执行分析。

四.优化的原理

1.DNS预解析原理

DNS预解析是一种优化技术，通过提前获取域名对应的IP地址，可以加速页面加载速度。当我们在网页中使用 `<link>` 标签的 `rel` 属性设置为 `dns-prefetch` 时，浏览器会提前进行 DNS 解析，以便在需要时能够更快地建立连接。

缓存解析结果：浏览器会将解析得到的 IP 地址存储在本地缓存中，以备将来使用。这样，在实际需要建立连接时，就可以直接使用缓存中的 IP 地址，当页面中包含大量外部资源或跨域请求时，可以加速后续的请求

2.CDN原理

CDN (全称 Content Delivery Network)，即内容分发网络，简单地说可以让用户就近获取所需内容，降低网络拥塞，提高用户访问响应速度和命中率。

原理：

DNS解析的过程：

- 1. 浏览器发起请求：**用户在浏览器中输入一个网址（比如 www.example.com）后按下回车，浏览器会向本地 DNS 服务器发送一个 DNS 解析请求。
- 2. 本地 DNS 服务器查询：**本地 DNS 服务器首先会检查自己的缓存中是否有对应域名的 IP 地址记录。如果有，就直接返回给浏览器；如果没有，本地 DNS 服务器会向根域名服务器发起查询请求。
- 3. 根域名服务器查询：**如果本地 DNS 服务器没有缓存记录，它会向根域名服务器发送一个请求，询问针对所查询域名的顶级域名服务器的地址。
- 4. 顶级域名服务器查询：**根域名服务器返回顶级域名服务器的地址给本地 DNS 服务器，本地 DNS 服务器再向顶级域名服务器发送请求，获取所查询域名的权威域名服务器地址。
- 5. 权威域名服务器查询：**本地 DNS 服务器最终向权威域名服务器发送请求，获取该域名对应的 IP 地址。
- 6. 返回结果：**权威域名服务器将查询结果（域名对应的 IP 地址）返回给本地 DNS 服务器，本地 DNS 服务器将结果缓存并返回给浏览器。
- 7. 浏览器访问网站：**拿到域名对应的 IP 地址后，浏览器会向这个 IP 地址发起 HTTP 请求，访问对应的网站

然而有了 CDN之后，情况发生了变化。

在web.com这个权威DNS服务器上，会设置一个CNAME别名，指向另外一个域名 www.web.cdn.com，返回给本地DNS服务器。

当本地DNS服务器拿到这个新的域名时，需要继续解析这个新的域名。这个时候，再访问的是**新域名的权威DNS服务器**，这是CDN自己的权威DNS服务器。在这个服务器上，还是会设置一个**CNAME**，指向另外一个域名，即CDN网络的全局负载均衡器。

接下来，本地DNS服务器去请求CDN的全局负载均衡器解析域名，全局负载均衡器会为用户选择一台**合适的**缓存服务器提供服务，选择的依据包括：

- 根据用户IP地址，判断哪一台服务器距用户最近；
- 用户所处的运营商；
- 根据用户所请求的URL中携带的内容名称，判断哪一台服务器上有用户所需的内容；
- 查询各个服务器当前的负载情况，判断哪一台服务器尚有服务能力。

基于以上这些条件，进行综合分析之后，全局负载均衡器会返回一台缓存服务器的IP地址。

本地DNS服务器缓存这个IP地址，然后将IP返回给客户端，客户端去访问这个边缘节点，下载资源。缓存服务器响应用户请求，将用户所需内容传送到用户终端。如果没有，则会向源服务器请求资源，并将资源缓存到节点服务器上。

缓存策略和刷新机制：

CDN 中的缓存更新和内容同步是确保用户获取最新内容的关键过程。以下是一些常见的缓存策略和刷新机制：

- 1. 缓存过期机制：**CDN 可以根据资源的缓存过期时间来决定何时刷新缓存。开发者可以在响应头中设置 **Cache-Control** 或 **Expires** 字段，告知 CDN 缓存的有效期限。一旦缓存过期，CDN 会向源服务器请求最新内容，并更新缓存。
- 2. 预热缓存：**在某些情况下，开发者可以通过预热缓存来提前加载和缓存资源，以应对突发的流量或活动。预热缓存可以在资源内容更新前提前将新内容缓存到节点服务器上，避免访问高峰时的延迟。
- 3. 自动刷新：**一些 CDN 提供自动刷新功能，可以根据规则或事件自动更新缓存。例如，当源站点内容发生变化时，CDN 可以自动检测并触发缓存刷新。这种自动化的刷新机制可以减轻开发者的管理负担。

适用场景

- 网站站点/应用加速

通俗讲就是static 内容加速，静态内容加速，如：html image js css 等

- 视音频点播/大文件下载分发加速

基本上都是视频点播，MP4、flv 等视频文件，例如国内的优酷、土豆、腾讯视频、爱奇艺都是一样。

- 视频直播加速

视频直播加速，流媒体切片、转码、码流转换等等。

熊猫TV、斗鱼、淘宝直播

- 移动应用加速

移动APP更新文件（apk文件）分发，移动APP内图片、页面、短视频、UGC等内容的优化加速分发。

ios、安卓端 APP、微信小程序、支付宝小程序等。

3.SSR

I.SPA vs SSR

SPA 一定是 CRS

- 单页应用程序 (SPA) 全称是：Single-page application，SPA应用是在客户端呈现的（术语称：CRS）。
 - SPA应用默认只返回一个空HTML页面，如：body只有<div id= "app" ></div>。
 - 而整个应用程序的内容都是通过 Javascript 动态加载，包括应用程序的逻辑、UI 以及与服务器通信相关的所有数据。
 - 构建 SPA 应用常见的库和框架有：React、AngularJS、Vue.js 等。

■ SPA的优点

□ 只需加载一次

- ✓ SPA应用程序只需要在第一次请求时加载页面，页面切换不需重新加载，而传统的Web应用程序必须在每次请求时都得加载页面，需要花费更多时间。因此，SPA页面加载速度要比传统 Web 应用程序更快。

□ 更好的用户体验

- ✓ SPA 提供类似于桌面或移动应用程序的体验。用户切换页面不必重新加载新页面
- ✓ 切换页面只是内容发生了变化，页面并没有重新加载，从而使体验变得更加流畅

□ 可轻松的构建功能丰富的Web应用程序

■ SPA的缺点

□ SPA应用默认只返回一个空HTML页面，不利于SEO（search engine optimization）

□ 首屏加载的资源过大时，一样会影响首屏的渲染

□ 也不利于构建复杂的项目，复杂 Web 应用程序的大文件可能变得难以维护

■ 静态站点生成(SSG) 全称是：Static Site Generate，是预先生成好的静态网站。

□ SSG 应用一般在构建阶段就确定了网站的内容。

□ 如果网站的内容需要更新了，那必须得重新再次构建和部署。

□ 构建 SSG 应用常见的库和框架有：Vue Nuxt、React Next.js 等。

■ SSG的优点：

□ 访问速度非常快，因为每个页面都是在构建阶段就已经提前生成好了。

□ 直接给浏览器返回静态的HTML，也有利于SEO

□ SSG应用依然保留了SPA应用的特性，比如：前端路由、响应式数据、虚拟DOM等

■ SSG的缺点：

□ 页面都是静态，不利于展示实时性的内容，实时性的更适合SSR。

□ 如果站点内容更新了，那必须得重新再次构建和部署。

■ 服务器端渲染全称是：Server Side Render，在服务器端渲染页面，并将渲染好HTML返回给浏览器呈现。

□ SSR应用的页面是在服务端渲染的，用户每请求一个SSR页面都会先在服务端进行渲染，然后将渲染好的页面，返回给浏览器呈现。

□ 构建 SSR 应用常见的库和框架有：Vue Nuxt、React Next.js 等（SSR应用也称同构应用）。

■ SSR的优点

□ 更快的首屏渲染速度

- ✓ 浏览器显示静态页面的内容要比 JavaScript 动态生成的内容快得多。
- ✓ 当用户访问首页时可立即返回静态页面内容，而不需要等待浏览器先加载完整个应用程序。

□ 更好的SEO

- ✓ 爬虫是最擅长爬取静态的HTML页面，服务器端直接返回一个静态的HTML给浏览器。
- ✓ 这样有利于爬虫快速抓取网页内容，并编入索引，有利于SEO。

□ SSR应用程序在 Hydration 之后依然可以保留 Web 应用程序的交互性。比如：前端路由、响应式数据、虚拟DOM等。

■ SSR的缺点

□ SSR 通常需要对服务器进行更多 API 调用，以及在服务器端渲染需要消耗更多的服务器资源，**成本高**。

□ **增加了一定的开发成本**，用户需要关心哪些代码是运行在服务器端，哪些代码是运行在浏览器端。

□ SSR 配置站点的**缓存**通常会比SPA站点要**复杂一点**。

补充缺点：

需要更多的服务器负载均衡。由于服务器增加了渲染HTML的需求，使得原本只需要输出静态资源文件的nodejs服务，新增了数据获取的IO和渲染HTML的CPU占用，如果流量突然暴增，有可能导致服务器down机，因此需要使用响应的缓存策略和准备相应的服务器负载。

II.原理

首先是浏览器请求URL，前端服务器接收到URL请求之后，根据不同的URL，前端服务器向后端服务器请求数据，请求完成后，前端服务器会组装一个携带了具体数据的HTML文本，并且返回给浏览器，浏览器得到HTML之后开始渲染页面，同时，浏览器加载并执行 JavaScript 脚本，给页面上的元素绑定事件，让页面变得可交互，当用户与浏览器页面进行交互，如跳转到下一个页面时，浏览器会执行 JavaScript 脚本，向后端服务器请求数据，获取完数据之后再次执行 JavaScript 代码动态渲染页面。

所谓**同构**，就是让一份代码，既可以在服务端中执行，也可以在客户端中执行，服务端渲染完成页面结构，客户端进行事件绑定和交互逻辑激活

那么如何实现同构呢？必须要从实践中出发。

基本的逻辑：

在服务器端渲染（SSR）React 应用时，通常会使用 `ReactDOM.renderToString` 将 React 组件渲染为 HTML 字符串，然后将该字符串发送给客户端。客户端接收到 HTML 字符串后，可以使用 `ReactDOM.hydrateRoot` 进行激活（hydrate）操作，将服务端生成的静态 HTML 转换为具有交互性的客户端应用。

注意的点：为了确保客户端激活的正确性，需要保持 `ReactDOM.renderToString` 和 `ReactDOM.hydrate` 或 `ReactDOM.hydrateRoot` 的参数一致，以便在客户端激活时能够正确地匹配并复用服务端生成的 HTML 结构。

ReactDOM.hydrateRoot原理：

具体的实现：

服务器端：

```
const express = require("express");
import React from "react";
import ReactDOM from "react-dom/server";
import { StaticRouter } from "react-router-dom/server";
import { Provider } from "react-redux";
import store from "../store/index";
import App from "../app.jsx";
const server = express();

// 静态资源的部署
server.use(express.static("build"));

server.get("/*", (req, res) => {
  // 就是服务器端渲染： / /about
  const AppHtmlString = ReactDOM.renderToString(
    <Provider store={store}>
```

```

    <StaticRouter location={req.url}>
      <App />
    </StaticRouter>
  </Provider>
);
res.send(`
  <!DOCTYPE html>
  <html lang="en">
    <head>
      <meta charset="UTF-8">
      <meta http-equiv="X-UA-Compatible" content="IE=edge">
      <meta name="viewport" content="width=device-width,
initial-scale=1.0">
      <title>Document</title>
    </head>
    <body>
      <h1>React18 + SSR</h1>
      <div id="root">${AppHtmlString}</div>
      <script src="/client/client_bundle.js"></script>
    </body>
  </html>
`);
});

server.listen(8080, () => {
  console.log("server start on 8080");
});

```

客户端:

```

import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import { Provider } from "react-redux";
import store from "../store/index";
import App from "../app.jsx";

```



```
// 在需要激活的模式下挂载应用 ( hydrateRoot )
ReactDOM.hydrateRoot(
  document.getElementById("root"),
  <Provider store={store}>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </Provider>
);
```

4.React路由懒加载原理

懒加载也就是异步加载，通俗地讲就是需要什么加载什么。SPA 项目，一个路由对应一个页面，如果不做处理，项目打包后，会把所有页面打包成一个文件，**当用户打开首页时，会一次性加载所有的资源**，造成首页加载白屏时间过长。

实现：

配置react路由时对组件进行 `React.lazy()` 加载，参数是一个回调，回调的返回值是一个`import()`方法实现异步加载。用 `Suspense` 组件包裹懒加载的组件。在Suspense组件可以用`fallback`参数来指定加载动画，这也是白屏性能优化的一个方法，不过更多是提高用户体验而不是提高本身的性能。

原理：

1.代码分割。当 Webpack 解析到 `import()` 语法时，会自动进行代码分割。代码分割指的是将打包后的 JavaScript 代码划分成更小、更独立的代码块，从而实现按需加载。=====》**原理：**核心是ECMA规范规定了`import()`的异步加载功能，本质上来说是调用`import`方法返回的是一个`promise`对象，创建方法内部通过动态创建`script`元素并监听其加载状态，根据是否加载成功来改变`promise`的状态是`fullfilled`还是`rejected`

2.`React.lazy()`：进一步处理这个promise对象。执行`import ()`方法返回的promise对象的`then`方法，如果promise对象已经Resolved了（说明模块已经加载完毕）则直接返回 `moduleObject.default`（动态加载的模块的默认导出），否则抛出异常，`React Suspense` 组件会捕获到这个异常

3.`Suspense`组件：捕获到异常之后会进行判断，如果处于 `pending` 状态，则会将该模块都渲染成 `fallback` 的值，一旦检测到 `resolve`则`Suspense`组件包裹的子组件会重新渲染。